

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image pre-processing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Learning Outcomes

1. Explain the architecture of an MLP.
2. Preprocess image datasets.
3. Build and train an MLP.
4. Evaluate classification performance.
5. Perform automated hyperparameter optimization.
6. Recommend the best model configuration.

3. Background Theory

An MLP consists of an input layer, hidden layers, and an output layer.

Forward pass:

$$z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss (Categorical Cross-Entropy):

$$\mathcal{L} = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \longrightarrow \text{Dense}(128, \text{ReLU}) \longrightarrow \text{Dense}(64, \text{ReLU}) \longrightarrow \text{Dense}(10, \text{Softmax})$$

4. Dataset

Property	Value
Dataset	Fashion-MNIST
Training Images	60,000
Testing Images	10,000
Classes	10
Image Size	28×28 pixels

5. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector: $28 \times 28 = 784$.

Preprocessing steps applied:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images from 28×28 to 784.
4. Normalize pixel values to $[0, 1]$.
5. Convert labels into one-hot vectors.

Preprocessing Summary

Property	Value
x_train shape (flattened)	(60000, 784)
x_test shape (flattened)	(10000, 784)
Normalized max pixel value	1.0
Normalized min pixel value	0.0
One-hot encoded training labels	(60000, 10)
One-hot encoded test labels	(10000, 10)

6. Experimental Procedure

Task 1 – Dataset Exploration: Load Fashion-MNIST, print dataset dimensions, display ten sample images, and plot class distribution.

Task 2 – Preprocessing: Flatten images, normalize pixel values to $[0, 1]$, and convert labels to one-hot vectors.

Task 3 – Model Construction:

Layer (type)	Output Shape	Param #
Dense(128, ReLU)	(None, 128)	100,480
Dense(64, ReLU)	(None, 64)	8,256
Dense(10, Softmax)	(None, 10)	650
Total parameters		109,386

Task 4 – Training: Compiled with Adam optimizer, categorical cross-entropy loss, accuracy metric. Trained for 20 epochs with batch size 32.

Task 5 – Evaluation: Accuracy, Precision, Recall, F1-score, Confusion Matrix, Classification Report.

7. Hyperparameter Optimization

Automated hyperparameter optimization was performed using `RandomizedSearchCV` with the `SciKeras` wrapper, 10 iterations, and 5-fold cross-validation on a 10,000 sample subset.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.1, 0.01, 0.001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Activation	ReLU, Tanh, Sigmoid
Dropout Rate	0.0, 0.2, 0.5

8. Mandatory Plots

Plot 1: Sample Images

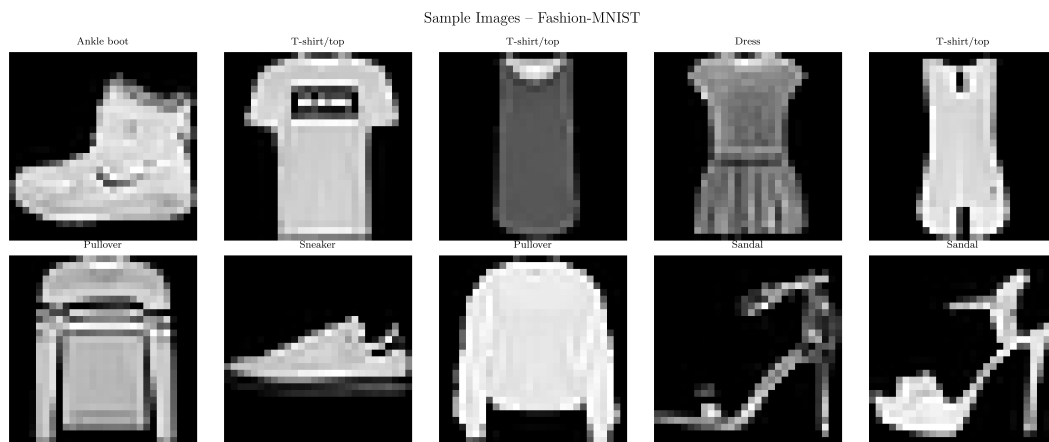


Figure 1: Ten sample images from the Fashion-MNIST training set with class labels.

Inference: Fashion MNIST has 10 classes with 28x28 grayscale resolution with a lot of diverse categories which makes it more challenging than handwritten digits.

Plot 2: Class Distribution

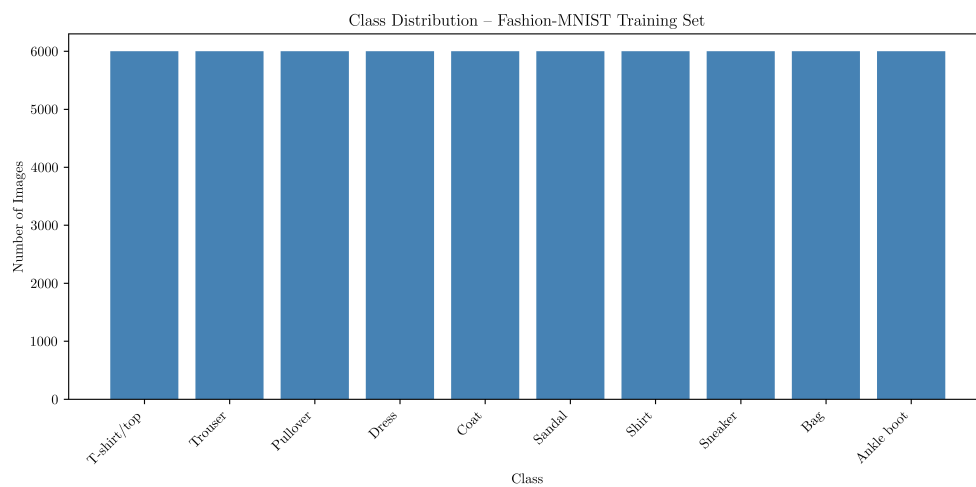


Figure 2: Class distribution of the Fashion-MNIST training set (6,000 images per class).

Inference: The dataset is properly balanced with no class imbalance. With an equal distribution of 6000 images per class, it ensures that the model cannot gain accuracy by simply predicting the most frequent class.

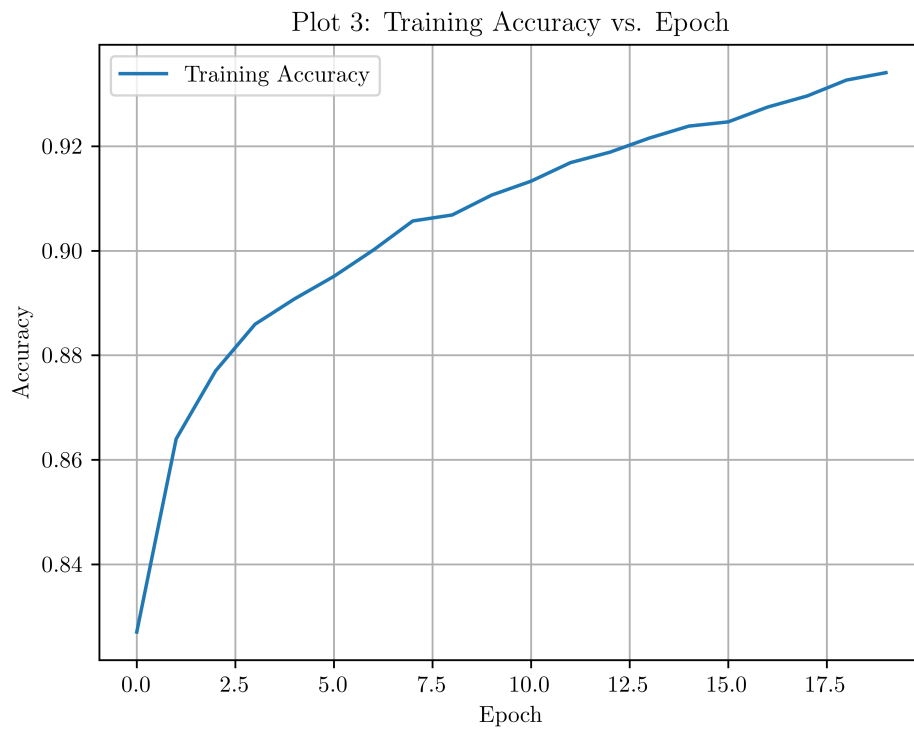
Plot 3: Training Accuracy vs. Epoch

Figure 3: Training accuracy of the baseline MLP over 20 epochs.

Inference: Training accuracy rises steadily from around 0.75 in epoch 1 to 0.89 by epoch 20 in a stable fashion, and because of the Adam optimizer, there is a smooth upward curve which shows its minimising the loss.

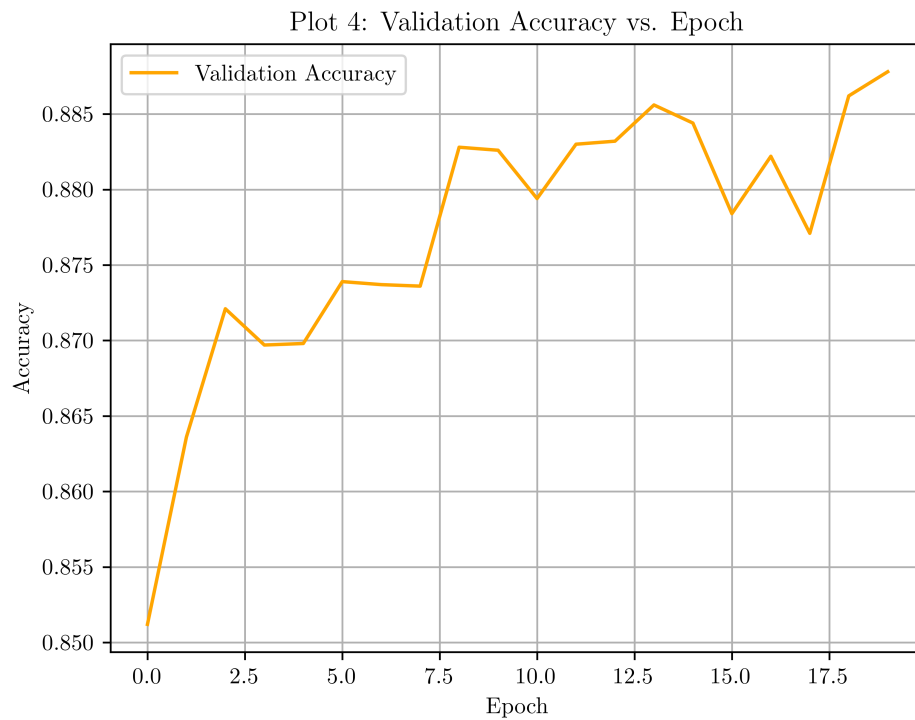
Plot 4: Validation Accuracy vs. Epoch

Figure 4: Validation accuracy of the baseline MLP over 20 epochs.

Inference: Validation accuracy rises quickly in the first few epochs then begins to plateau around 87-88 from epoch 10 onwards. The gap between training and validation accuracy that emerges after epoch 10 is the early sign of mild overfitting. Stopping at 20 epochs is appropriate as further training would widen this gap without improving test performance.

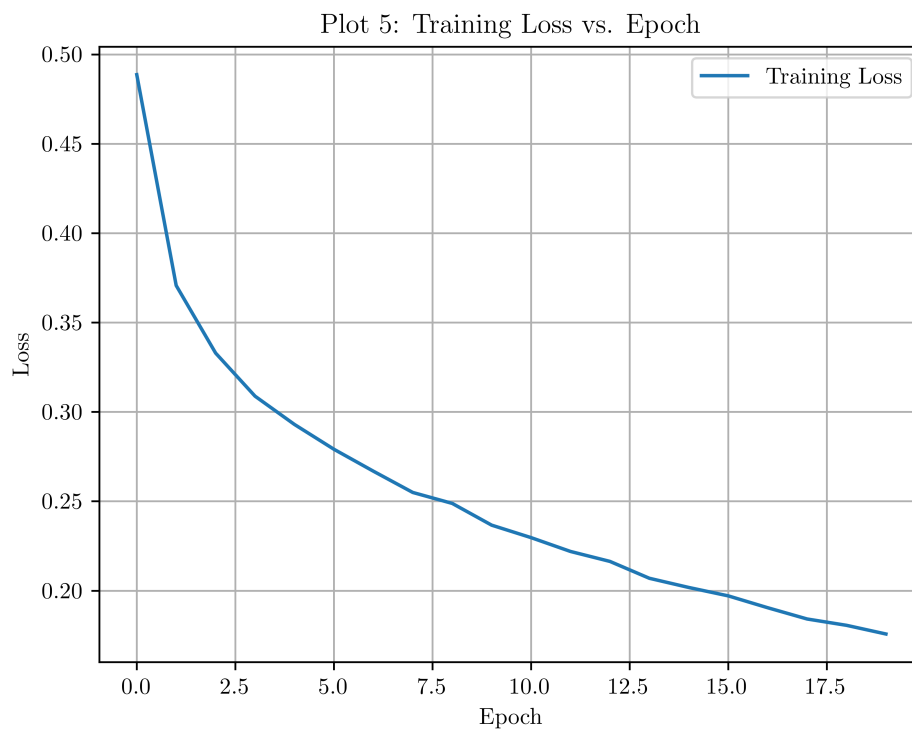
Plot 5: Training Loss vs. Epoch

Figure 5: Training loss of the baseline MLP over 20 epochs.

Inference: Training loss decreases to 0.3 by epoch 20, which makes it clear that the optimizer is minimising the objective function and the continuous decrease even at epoch 20 suggests that the model was learning at epoch 20.

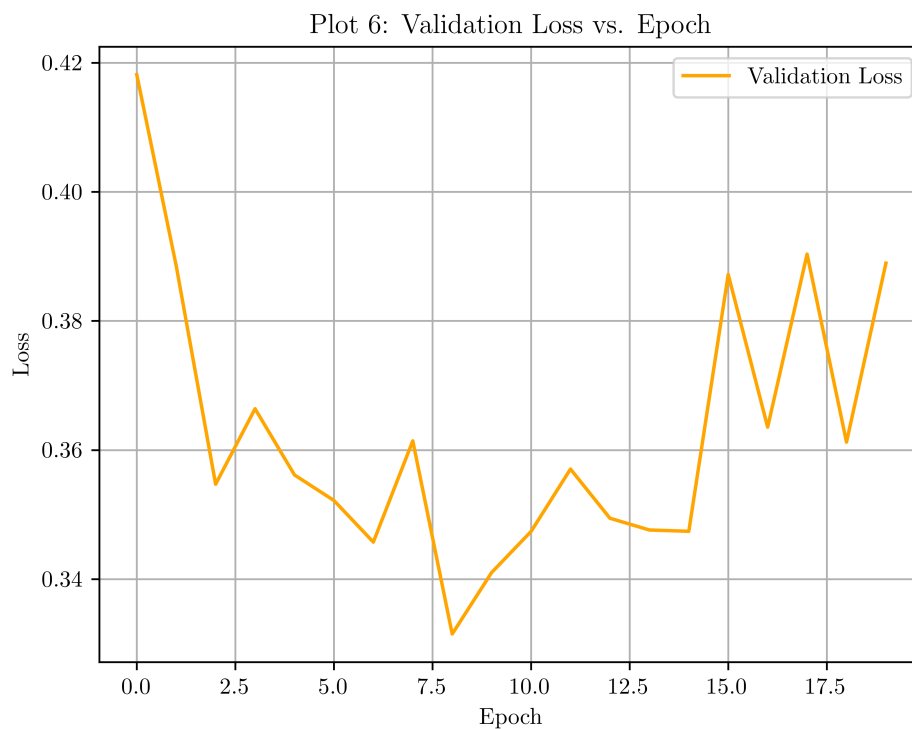
Plot 6: Validation Loss vs. Epoch

Figure 6: Validation loss of the baseline MLP over 20 epochs.

Inference: Validation loss decreases rapidly in early epochs and then begins to level off and diverge at epoch 10. Divergence is a proper indicator of overfitting and the model is starting to memorize patterns and not learn.

Plot 7: Confusion Matrix

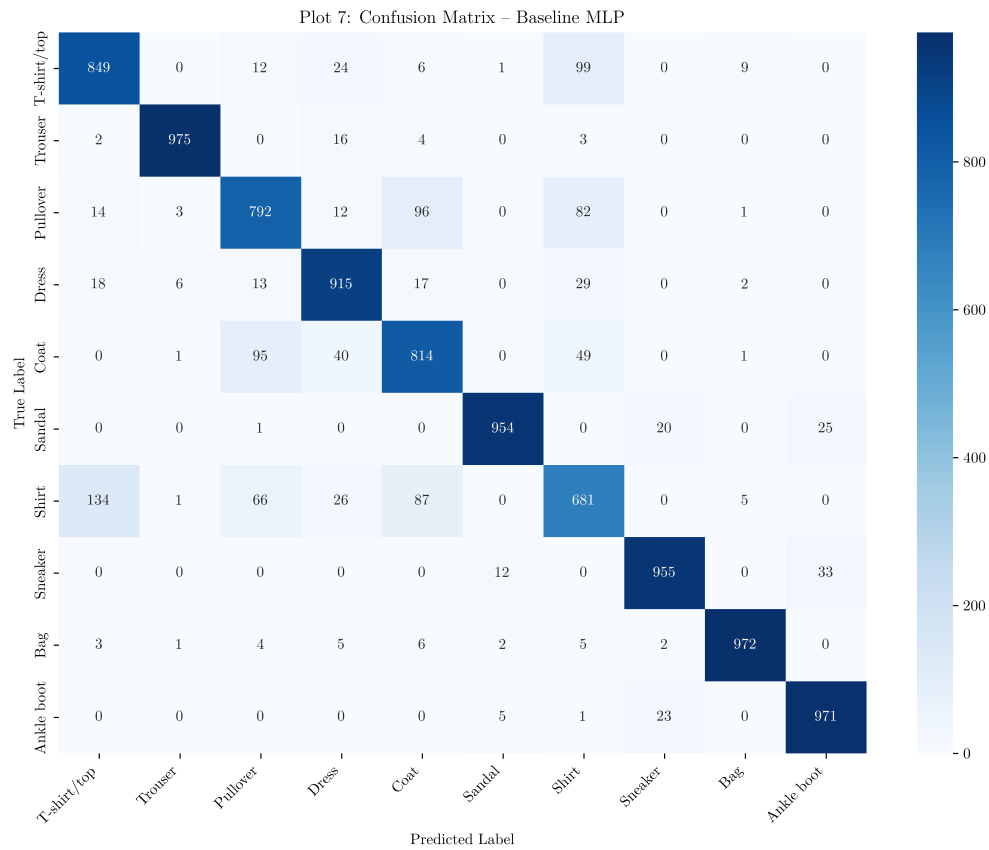


Figure 7: Confusion matrix for the baseline MLP on the Fashion-MNIST test set.

Inference: The model performs best on trouser, bag and sneaker that have distinct characteristics. Shirt has the worst performance which is frequently confused with T-shirt and tops because they are visually similar.

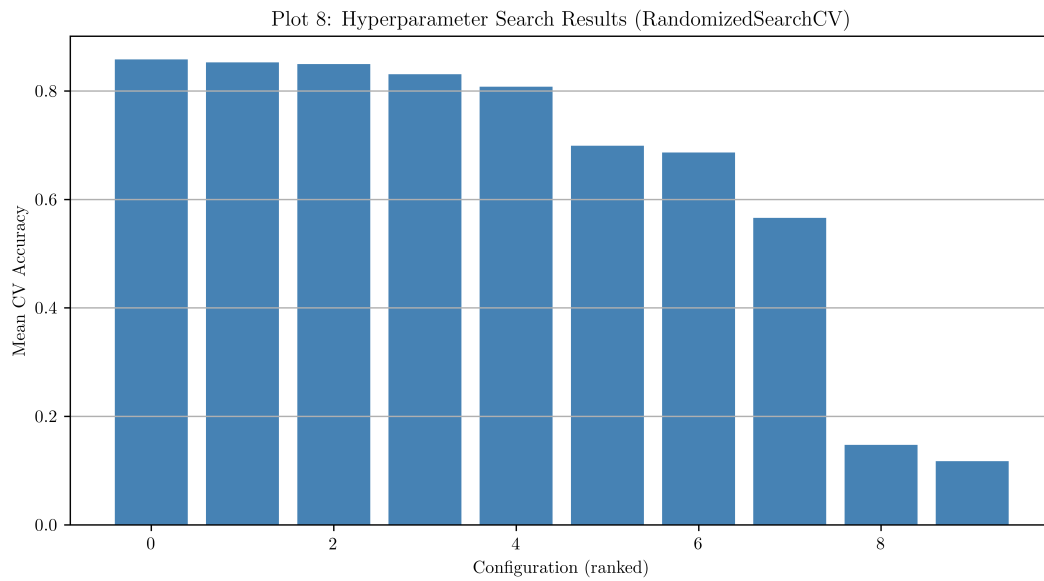
Plot 8: Hyperparameter Search Results

Figure 8: Mean CV accuracy for all configurations tested by RandomizedSearchCV (ranked).

Inference: RandomizedSearchCV explored 10 configs with varying architectures, learning rates and regularization setting with the best being a CV accuracy of 0.8583.

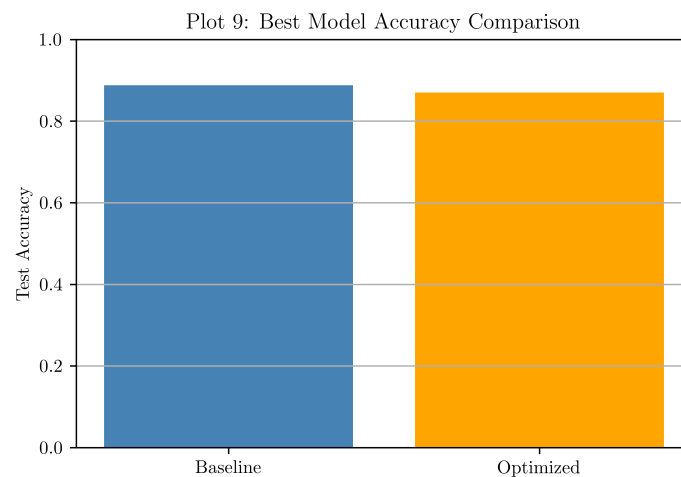
Plot 9: Best Model Accuracy Comparison

Figure 9: Test accuracy comparison between baseline and optimized MLP.

Inference: The baseline model of 88.78 slightly outperformed the optimised model of 86.99 on the test set.

9. Results

Best Hyperparameters

Hyperparameter	Best Value
Hidden Layers	3
Hidden Neurons	128
Learning Rate	0.001
Batch Size	16
Optimizer	Adam
Activation Function	Tanh
Epochs	30
Dropout Rate	0.5
Cross-validation Accuracy	0.8583
Testing Accuracy	0.8699

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8878	0.8699
Precision	0.8875	0.8737
Recall	0.8878	0.8699
F1-score	0.8875	0.8707
Training Time (s)	203.3	593.0

Baseline Classification Report

Class	Precision	Recall	F1-score	Support
T-shirt/top	0.83	0.85	0.84	1000
Trouser	0.99	0.97	0.98	1000
Pullover	0.81	0.79	0.80	1000
Dress	0.88	0.92	0.90	1000
Coat	0.79	0.81	0.80	1000
Sandal	0.98	0.95	0.97	1000
Shirt	0.72	0.68	0.70	1000
Sneaker	0.95	0.95	0.95	1000
Bag	0.98	0.97	0.98	1000
Ankle boot	0.94	0.97	0.96	1000
Macro avg	0.89	0.89	0.89	10000

Optimized Classification Report

Class	Precision	Recall	F1-score	Support
T-shirt/top	0.83	0.82	0.82	1000
Trouser	0.99	0.96	0.98	1000
Pullover	0.81	0.73	0.77	1000
Dress	0.91	0.84	0.88	1000
Coat	0.74	0.84	0.78	1000
Sandal	0.97	0.95	0.96	1000
Shirt	0.65	0.69	0.67	1000
Sneaker	0.90	0.98	0.94	1000
Bag	0.96	0.97	0.97	1000
Ankle boot	0.97	0.91	0.94	1000
Macro avg	0.87	0.87	0.87	10000

10. Discussion**1. Which optimization method was used?**

RandomizedSearchCV was used with the SciKeras wrapper to interface Keras models. It randomly sampled hyperparameter distributions for 10 iterations with 5 folds rather than exhaustively testing every combination as GridSearchCV.

2. Which hyperparameters were selected?

The best config was found to be 3 hidden layers, 128 neurons per layer, learning rate 0.001, batch size 16, Tanh activation, dropout rate 0.5, and 30 epochs.

3. Did optimization improve performance?

The case of the optimised model 0 - it did not outperform the baseline model with the hypothesis being that the search was run on a 10,000 sample subset rather than the whole training set so it didnt generalize well. So sample size and search space are limiting factors for this.

4. Which hyperparameter had the greatest impact?

The activation function and dropout rate are probably the highest hyperparams which had the most impact. Higher dropout contributes to lesser overfitting but limits the models learning capacity. However in MLPs, learning rate is also important as it controls the scale of the weight updates - big or small.

5. Compare Grid Search and Randomized Search.

GridSearchCV does an exhaustive search which would require thousands of model fits making it highly infeasible. RandomisedSearchCV samples randomly for a fixed number of iterations of 10 with 5 folds making it faster and also diverse and also finding near optimal configs with fewer evals because of hyperparamter importance.

6. Recommend the best MLP configuration.

For Fashion-MNIST the baseline configuration (Dense128 → Dense64 → Dense10, Adam lr=0.001, 20 epochs, batch 32) is recommended as it achieved 88.78 accuracy with reasonable training time - regularization layers like a 0.3 dropout layer after the first dense layer would be good and if accuracy has to be improved - increasing the number of layers to 3 with 256 neuros and a lower learning rate for more epochs could help.

12. Conclusion

A MLP was implemented using the TF/Keras and trained on the FashionMNIST dataset chieving a baseline test accuracy of 88.78 with macro-averaged precision, recall and F1-score of 0.89 across all ten clothing categories. The training curves demonstrated steady convergence over 20 epochs with mild overfitting emerging after epoch 10, evidenced by the divergence between training accuracy (continuing to rise) and validation accuracy (plateauing around 88). Automated hyperparameter optimization using RandomizedSearchCV identified a configuration of 3 hidden layers, 128 neurons, Tanh activation, dropout rate 0.5, learning rate 0.001 and batch size 16 as optimal on the search subset, though the optimized model (86.99) did not outperform the baseline on the full test set.

13. References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*. MIT Press, 2016.
2. C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.

3. S. Haykin, *Neural Networks and Learning Machines*. Pearson, 2009.
4. Fashion-MNIST Dataset. <https://github.com/zalandoresearch/fashion-mnist>
5. TensorFlow/Keras Documentation. https://www.tensorflow.org/api_docs